

Pitch-Dokument: Programmieren 1 – System Recovery

Abstract

Ein Digital Educational Escape Room für Programmieren 1, in dem Studierende eine beschädigte Informatik-Forschungsstation reparieren, indem sie Arrays programmieren, sichtbar machen und manipulieren. Der Raum macht Array-Konzepte über konkrete Objekte in der Spielwelt erfahrbar: Werte erscheinen als Kristalle, Module als Chips und 2D-Arrays als begehbare Raster. Java wird als Eingabe am Terminal genutzt; der geschriebene Code erzeugt unmittelbar sichtbare Veränderungen im Spiel. Am Ende haben die Studierenden zentrale Array-Operationen als zusammenhängenden Programmablauf erlebt: Arrays erstellen, füllen, verändern, iterieren, sortieren und in zwei Dimensionen durchsuchen.

1. Kurzprofil

Arbeitstitel: System Recovery

Zielgruppe / Veranstaltungskontext: Informatik, 1. Semester; Einsatz nach der Array-Einführung in Programmieren 1 als Praktikum, Selbststudium oder Vertiefung

Dauer: ca. 60-90 Minuten; für 60 Minuten können Sortier- und Finalaufgaben gekürzt werden

Gruppengröße: 1-2 Personen

Fachlicher Schwerpunkt: Arrays, Indizes, `null`, Iteration, `for`, `for-each`, Sortieren, Bubble Sort, 2D-Arrays und verschachtelte Schleifen

2. Erwartete Lernergebnisse

- Die Spielenden erstellen Arrays mit festgelegter Größe und befüllen sie über Indizes.
- Die Spielenden greifen gezielt auf Array-Elemente zu und verändern einzelne Werte.
- Die Spielenden unterscheiden Array-Länge und tatsächlich belegte Elemente.
- Die Spielenden verwenden `null`, um leere oder entfernte Elemente zu modellieren.
- Die Spielenden iterieren über Arrays mit `for` und `for-each`.
- Die Spielenden zählen Elemente anhand einer Bedingung.
- Die Spielenden sortieren Arrays und verstehen die Grundidee von Bubble Sort.
- Die Spielenden erstellen und nutzen 2D-Arrays mit Zeilen- und Spaltenindizes.
- Die Spielenden durchsuchen 2D-Arrays mit verschachtelten Schleifen.

3. Spielrahmen

Narratives Szenario:

Die Studierenden betreten eine beschädigte Informatik-Forschungsstation. Das zentrale System ist ausgefallen, mehrere Subsysteme sind gesperrt und die Datenstrukturen der Station müssen Schritt für Schritt wiederhergestellt werden.

Ziel der Spielenden:

Sie reparieren die Station, indem sie Arrays im Terminal programmieren, deren Wirkung in der Spielwelt beobachten und die sichtbaren Datenstrukturen anschließend für Rätsel, Sortieraufgaben und Türschlösser nutzen.

Spielmechanik im Überblick:

Die Spielenden geben Java-Code in einen Array-Compiler ein (Arrays erstellen und befüllen), sehen dadurch Objekte in der Spielwelt erscheinen (Materialisierung von Datenstrukturen), verändern einzelne Slots oder Objekte (Indexzugriff, Zuweisung, `null`), steuern Scanner und Roboter über Schleifen (Iteration mit `for` und `for-each`), sortieren Datenobjekte manuell und algorithmisch (Sortierlogik, Bubble Sort) und durchsuchen Rasterräume (2D-Arrays, verschachtelte Schleifen).

Ton und Atmosphäre:

Science-Fiction-nahes Informatik-Setting mit beschädigter Forschungsstation, Terminalsystemen, Speichern, Modulen, Scannern und Robotern.

Zentrales Designprinzip:

Jedes Rätsel folgt idealerweise derselben Lernschleife:

1. Code schreiben
2. Datenstruktur erscheint
3. Objekte manipulieren
4. Ergebnis interpretieren
5. Programmier-Schloss lösen

4. Ablaufübersicht

Phase	Funktion im Spiel	Raum / Interface	Fachlicher Zweck	Spielergebnis	Lernergebnis
Raum 1	Energie-Array materialisieren	Terminal und leerer Array-Speicher	Array erstellen, Größe festlegen, Werte über Indizes setzen	Fünf Energiekristalle erscheinen und öffnen die erste Tür.	Die Spielenden verstehen Arrays als feste Speicherplätze mit indexbasiertem Zugriff.
Raum 2	Defekten Modulspeicher reparieren	Modulchips in einem String-Array	Datentypen, Elementänderung, <code>null</code> , <code>length</code>	Der beschädigte GPU-Chip wird entfernt, ohne die Speichergröße zu verändern.	Die Spielenden unterscheiden leere Elemente von der Länge des Arrays.
Raum 3	Inventarscanner aktivieren	Scanner läuft über Modulslots	<code>for-each</code> , Zählen, <code>null</code> prüfen	Der Scanner zählt vier aktive Module.	Die Spielenden zählen nur Elemente, die eine Bedingung erfüllen.
Raum 4	Transportroboter steuern	Förderbänder mit Datenpaketen	klassische <code>for</code> -Schleife, Indexvariable, <code>array.length</code>	Der Roboter nimmt alle Pakete nacheinander auf.	Die Spielenden verknüpfen <code>i</code> mit Position und <code>array[i]</code> mit dem Wert an dieser Position.
Raum 5	Chaotischen Datenspeicher ordnen	Bewegliche Datenkristalle	Sortieren und Vergleichen von Werten	Unsortierte Sicherheitsdaten werden aufsteigend geordnet.	Die Spielenden erleben Sortieren zunächst als sichtbare Ordnungshandlung.
Raum 6	Bubble-Sort-Maschine reparieren	mechanische Vergleichsmaschine	Bubble Sort, Tauschlogik, verschachtelte Schleifen	Die Maschine sortiert Schritt für Schritt durch Vergleichen benachbarter Werte.	Die Spielenden verstehen Vergleich, Tausch mit temporärer Variable und wiederholte Durchläufe.
Raum 7	Datenarchiv rekonstruieren	Archiv mit verschiedenen Datensätzen	Arrays verschiedener Datentypen	Zahlen, Texte und Wahrheitswerte werden passenden Arrays zugeordnet.	Die Spielenden erkennen, dass Arrays typgebunden sind.
Raum 8	2D-Speicher aufbauen	begehbare Raster mit Zeilen und Spalten	2D-Arrays, zwei Indizes, Platzierung	Objekte erscheinen an vorgegebenen Rasterpositionen.	Die Spielenden verbinden <code>array[zeile][spalte]</code> mit Koordinaten in der Spielwelt.

Phase	Funktion im Spiel	Raum / Interface	Fachlicher Zweck	Spielergebnis	Lernergebnis
Raum 9	Suchroboter einsetzen	Rasterkarte mit Objekten	verschachtelte <code>for</code> -Schleifen, 2D-Iteration	Der Roboter durchsucht jede Kachel und sammelt Batterien ein.	Die Spielenden verstehen, wie ein 2D-Array vollständig durchlaufen wird.
Finale	Zentrales Rechenzentrum wiederherstellen	drei Subsysteme und finales Türschloss	Synthese aller Array-Konzepte	Energieversorgung, Modulkontrolle und Rechenzentrumskarte werden repariert.	Die Spielenden kombinieren Sortieren, Zählen und 2D-Suche zu einem Gesamtprogramm.

5. Rätselsteckbriefe

Raum 1: Die Materialisierungskammer

Spielhandlung:

Die Studierenden betreten eine Kammer mit einer defekten Materialisierungsmaschine. Das System meldet, dass ein Energie-Array mit fünf Speicherplätzen fehlt.

Was die Spielenden konkret machen:

Sie geben Java-Code in ein Terminal ein, um ein Array zu erzeugen und anschließend mit Energiewerten zu befüllen. Aus `int[] energie = new int[5];` entstehen fünf leere Slots; aus Zuweisungen wie `energie[2] = 80;` werden sichtbare Energiekristalle.

Fachlicher Zusammenhang:

Der Raum macht Array-Größe, Indexpositionen und Wertzuweisung sichtbar. Besonders wichtig ist der Zusammenhang zwischen Slotnummer und Array-Index.

Lernfunktion:

Grundaufbau von Arrays und indexbasierter Zugriff. Der erste Raum legt das zentrale Prinzip des Escape Rooms fest: Code verändert sichtbar die Spielwelt.

Ergebnis / Unlock:

Das Energie-Array ist vollständig gefüllt. Die Tür verlangt einen gezielten Zugriff, zum Beispiel den Wert an Speicherplatz `2`.

Raum 2: Der defekte Modulspeicher

Spielhandlung:

Ein beschädigter Computerspeicher enthält fünf Systemmodule: CPU, RAM, GPU, SSD und NETWORK. Die GPU ist defekt und darf nicht weiter verwendet werden.

Was die Spielenden konkret machen:

Sie erzeugen ein `String[]`, befüllen es mit Modulnamen und markieren das defekte Element mit `module[2] = null;`. In der Spielwelt verschwindet der GPU-Chip, der Slot bleibt aber sichtbar leer.

Fachlicher Zusammenhang:

Der Raum verbindet String-Arrays, Elementänderung und `null`. Zusätzlich wird `module.length` genutzt, um zu zeigen, dass die Array-Länge weiterhin `5` beträgt, obwohl nur vier Module aktiv sind.

Lernfunktion:

Unterscheidung zwischen Speicherplatz und Inhalt. `null` wird nicht als "Array ist kleiner geworden" verstanden, sondern als leerer Wert an einer vorhandenen Position.

Ergebnis / Unlock:

Der Modulspeicher ist repariert und die Studierenden kennen die weiterhin gültige Speichergröße.

Raum 3: Der Inventarscanner

Spielhandlung:

Ein Scanner muss feststellen, wie viele funktionierende Module noch vorhanden sind.

Was die Spielenden konkret machen:

Sie lassen den Scanner nacheinander über alle Elemente laufen und zählen nur Einträge, die nicht `null` sind. Dazu passt eine `for-each`-Schleife mit Bedingung: Wenn ein Modul nicht leer ist, wird der Zähler erhöht.

Fachlicher Zusammenhang:

Der Raum führt Iteration ohne expliziten Index ein. Die Spielenden sehen, dass `for-each` gut geeignet ist, wenn jedes Element betrachtet, aber keine Position verändert werden muss.

Lernfunktion:

Zählen mit Bedingung und Umgang mit `null` beim Iterieren.

Ergebnis / Unlock:

Das Scanner-Schloss akzeptiert `4`, weil vier Module aktiv sind.

Raum 4: Das Transportlager

Spielhandlung:

Ein Transportroboter muss fünf Datenpakete von Förderbändern aufnehmen.

Was die Spielenden konkret machen:

Sie steuern den Roboter über eine klassische `for`-Schleife. Im Spiel wird sichtbar, wie `i = 0`, `i = 1` usw. nacheinander auf die Positionen im Array zeigen und `pakete[i]` den jeweiligen Wert liefert.

Fachlicher Zusammenhang:

Der Raum macht die Indexvariable als Steuergröße sichtbar. `array.length` begrenzt die Schleife und verhindert, dass außerhalb des Arrays zugegriffen wird.

Lernfunktion:

Verständnis von `for`, Index und Zugriffsmuster `array[i]`.

Ergebnis / Unlock:

Der Roboter transportiert alle Pakete in der korrekten Reihenfolge.

Raum 5: Der chaotische Datenspeicher

Spielhandlung:

Ein Sicherheitssystem enthält unsortierte Datenkristalle. Es funktioniert erst, wenn die Werte aufsteigend sortiert sind.

Was die Spielenden konkret machen:

Sie verschieben Datenkristalle manuell in die richtige Reihenfolge, zum Beispiel von [73] [12] [45] [8] [31] zu [8] [12] [31] [45] [73] .

Fachlicher Zusammenhang:

Sortieren wird zunächst als sichtbare Handlung eingeführt: Werte vergleichen, Ordnung herstellen und Positionen verändern. Erst danach wird die Frage vorbereitet, wie ein Programm dieselbe Ordnung automatisch erzeugt.

Lernfunktion:

Vorbereitung algorithmischen Denkens. Die Studierenden übertragen eine manuelle Sortierhandlung später in Code.

Ergebnis / Unlock:

Der Datenspeicher ist geordnet und die Station fordert eine algorithmische Sortierlösung.

Raum 6: Die Bubble-Sort-Maschine

Spielhandlung:

Die Studierenden finden eine mechanische Sortiermaschine, die jeweils zwei benachbarte Werte vergleichen kann.

Was die Spielenden konkret machen:

Sie entscheiden bei Vergleichspaaren, ob getauscht werden muss. Wenn links größer als rechts ist, wählen sie "Tauschen"; andernfalls bleibt die Reihenfolge erhalten. Anschließend ergänzen sie im vorbereiteten Bubble-Sort-Code die zentrale Bedingung `array[j] > array[j + 1]` .

Fachlicher Zusammenhang:

Der Raum zeigt Bubble Sort als wiederholten Vergleich benachbarter Elemente. Sichtbar werden die innere Schleife, die Tauschlogik und die temporäre Variable.

Lernfunktion:

Algorithmisches Verständnis statt bloßer Codeabschrift. Die Spielenden sehen, warum mehrfach über das Array gelaufen wird.

Ergebnis / Unlock:

Die Sortiermaschine kann unsortierte Werte automatisch in aufsteigende Reihenfolge bringen.

Raum 7: Das Datenarchiv

Spielhandlung:

Im Archiv liegen verschiedene Datensätze, die dem passenden Array-Typ zugeordnet werden müssen.

Was die Spielenden konkret machen:

Sie entscheiden, ob Zahlen, Texte oder Ja/Nein-Werte als `int[]`, `String[]` oder `boolean[]` modelliert werden. In der Welt erscheinen dazu Energiewerte, Modulchips oder Statusanzeigen.

Fachlicher Zusammenhang:

Der Raum macht Typbindung von Arrays sichtbar: Ein Array enthält Elemente eines bestimmten Typs.

Lernfunktion:

Festigung verschiedener Array-Typen und Transfer der Datentypen aus den Grundlagen auf Array-Strukturen.

Ergebnis / Unlock:

Die Archivdaten sind korrekt rekonstruiert.

Raum 8: Der zweidimensionale Speicher

Spielhandlung:

Die Studierenden betreten einen Lagerraum, dessen Boden als Raster aufgebaut ist. Das Spielfeld entspricht einem 2D-Array.

Was die Spielenden konkret machen:

Sie erzeugen mit `int[][] lager = new int[3][4];` ein Raster mit drei Zeilen und vier Spalten. Anschließend platzieren sie Objekte über zwei Indizes, etwa Batterie, Schlüssel oder Computer.

Fachlicher Zusammenhang:

Der Raum übersetzt `array[zeile][spalte]` direkt in eine Position im Raum. Zeilen- und Spaltenindizes werden dadurch räumlich erfahrbar.

Lernfunktion:

Verständnis von 2D-Arrays, Koordinaten und Zugriff mit zwei Indizes.

Ergebnis / Unlock:

Das Lager-Raster ist korrekt erzeugt und mit Objekten an den richtigen Positionen befüllt.

Raum 9: Der Suchroboter

Spielhandlung:

Ein Roboter muss in einer Karte alle Batterien finden. Die Karte liegt als 2D-Array vor.

Was die Spielenden konkret machen:

Sie schreiben verschachtelte `for`-Schleifen, die alle Zeilen und darin alle Spalten durchlaufen. Währenddessen untersucht der Roboter jede Kachel sichtbar nacheinander.

Fachlicher Zusammenhang:

Die äußere Schleife durchläuft die Zeilen, die innere Schleife die Spalten. Eine Bedingung wie `map[i][j] == 1` erkennt Batterien.

Lernfunktion:

Systematisches Durchsuchen eines 2D-Arrays und Verständnis verschachtelter Iteration.

Ergebnis / Unlock:

Der Roboter sammelt alle Batterien ein und aktiviert den Zugang zum Rechenzentrum.

Finale: Das zentrale Rechenzentrum

Spielhandlung:

Das zentrale System der Forschungsstation ist noch gesperrt. Drei Subsysteme müssen repariert werden: Energieversorgung, Modulkontrolle und Rechenzentrumskarte.

Was die Spielenden konkret machen:

Sie kombinieren vorherige Lösungsstrategien: ein Energie-Array sortieren, aktive Module mit `for-each` zählen und in einem 2D-Array Batterien mit verschachtelten Schleifen finden. Das finale Türschloss verlangt die entscheidenden Codefragmente, die den Systemzustand wiederherstellen.

Fachlicher Zusammenhang:

Das Finale bündelt Sortieren, Zählen, `null`-Prüfung, 2D-Zugriff und verschachtelte Schleifen.

Lernfunktion:

Transfer und Synthese. Die Studierenden wenden Array-Konzepte nicht isoliert, sondern als zusammenhängende Problemlösung an.

Ergebnis / Unlock:

Die Forschungsstation wird wiederhergestellt und der Escape Room endet mit erfolgreicher System Recovery.